COP 3502C Programming Assignment # 6

Heap

Please read all the pages before starting to write your code

Please include the following commented lines at the beginning of your code to declare your authorship of the code:

/* COP 3502C PA6

This program is written by: Your Full Name */

Compliance with Rules: *The UCF Golden Rules apply to this assignment and its submission. In addition, all assignment policies outlined in the syllabus also apply here. The instructor or TA may ask any student to explain any part of their code to verify authorship and ensure clarity. Sharing this assignment description or your code, in whole or in part, with anyone or anywhere is a violation of course policy. Likewise, using code obtained from any outside source, including AI tools, will be considered cheating. Your primary resources should be class notes, lectures, labs, and TAs. These rules are in place to ensure fairness, maintain academic integrity, and give every student the opportunity to develop their own problem-solving and coding skills.*

Deadline:

Please check Webcourses for the official assignment deadline. Assignments submitted by email will not be graded, and such emails will not receive a response in accordance with course policy.

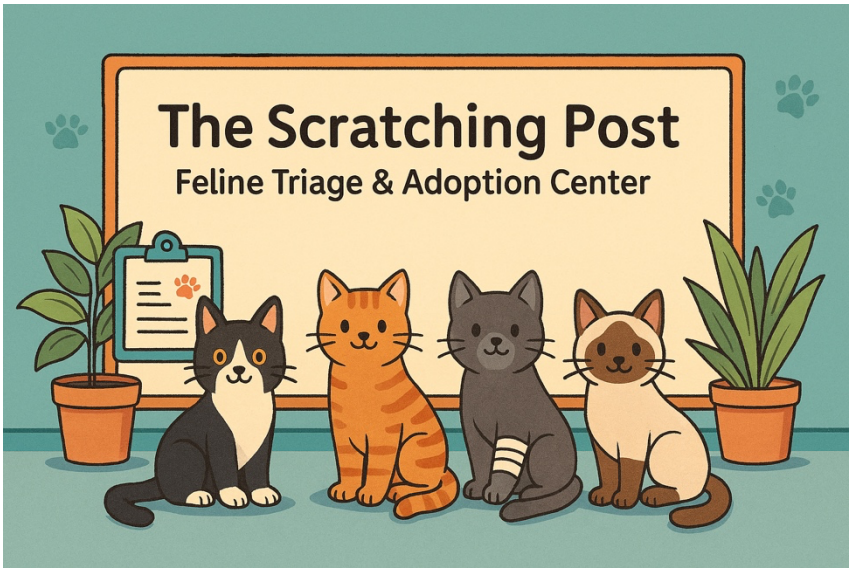
What to do if you need clarification on the problem?

If you need clarification on the problem, a discussion thread will be created on Webcourses, and you are strongly encouraged to post your questions there. You may also ask questions on Discord; if TAs are available, they can respond, and other students might also contribute, helping you get answers more quickly. For further clarification on the requirements, you are welcome to visit the TAs or the instructor during office hours.

How to get help if you're stuck?

As stated in the course policy, office hours are the primary place to seek help. While we may occasionally respond to emails, debugging requests are best addressed during office hours rather than through email.

PA6: The Scratching Post
Feline Triage & Adoption Priority System



Objective:

In this assignment, you will practice:

- Array-based binary heap with mode-dependent comparator.
- Priority-based extraction using a unified service command.
- Key recomputation + O(n) heapify on mode changes.
- Heap-based priority queue with linear search for name-based updates and removals.
- Clean dynamic memory management and deterministic output formatting.

Background Story

Welcome to The Scratching Post, the largest feline triage and adoption center in Mew York City. From sun-kissed strays to elderly long-timers, the center cares for thousands of cats each year.

But with rising rescue numbers, the staff can no longer manually decide: which cats need urgent medical care, which cats are ideal for same-day adoption, which cats should temporarily remain in quarantine, how shifting conditions (age, injuries, health changes) affect priority, and how special campaigns (like Featured Breed Week) influence adoptability.

They now turn to you, the center's most trusted systems engineer, to implement a dynamic, heap-driven priority engine capable of:

- Ranking cats in Adoption Mode (max-heap),
- Ranking cats in Triage Mode (min-heap),
- Automatically recalculating priorities when modes change,
- Supporting cat attribute updates,
- Enforcing Quarantine Protocols,
- Respecting Featured Breed bonuses,
- And handling all shelter operations through a consistent command language.

Your system must be fast, memory-safe, and perfectly deterministic, no surprises allowed when real cats' wellbeing is on the line.

System Overview

The system maintains a live collection of cats. For each cat, the system computes a priority key based on the current mode, stores the cat in a heap. Cats are identified by name using a linear search over the heap array when updates or removals are requested. Two global modes exist: 1) **`Adoption`** - A max-heap ranking cats with the highest adoptability score at the top, and 2) **`Triage`** - A min-heap ranking cats with the lowest (most urgent) triage score at the top. **Default mode is `Adoption`.** Switching modes forces a full key recompute and heap rebuild. Please check the provided structure named `Cat` to see the attributes a `Cat` type object will store. Following are the scoring formulae for the two global modes (`Adoption` and `Triage`):

Mode 1: Adoption (Max-Heap)

```
base = 1.6 * friendliness + 1.1 * health - 0.7 * age
mult = (breed == featured ? alpha : 1.0)
adoption_key = base * mult + (-1e-6 * arrival_id)
```

Higher is better. So, higher `key` ranks higher; tie-breakers ⇒ lexicographically smaller `name`, then smaller `arrival_id`.

Mode 2: Triage (Min-Heap)

```
triage_key = (100-health)*2.0 + max(0, age-12)*1.0 - 0.05*friendliness
```

Lower = more urgent. So, lower `key` ranks higher; tie-breakers ⇒ lexicographically smaller `name`, then smaller `arrival_id`.

Serving Cats:

The system provides a unified **SERVE** command that removes and processes the highest-priority cat based on the current mode.

- In ADOPTION mode, SERVE adopts the highest-priority non-quarantined cat (max-heap behavior).
- In TRIAGE mode, SERVE treats the most urgent cat (min-heap behavior).

Switching modes changes the interpretation of priority but does not change the underlying heap structure.

Additionally, a quarantine protocol needs to be implemented. A quarantined cat (`quarantine = 1`) cannot be adopted. However, a quarantined cat can be treated in TRIAGE mode. In ADOPTION mode: the system extracts top cats until it finds a non-quarantined one, skipped quarantined cats are temporarily stored and reinserted afterwards. If only quarantined cats exist, the system will print: "`No adoptable cats available.`" Quarantine does not affect numeric priority values, only adoption eligibility.

Commands and Input Specifications

The first line:

```
q    (number of commands)
```

Then q lines of:

- **Add a cat:** `ADD <name> <breed> <age> <friendliness> <health>`
- **Update a cat:** `UPDATE <name> <FIELD> <value>`
- **Remove a cat:** `REMOVE <name>`
- **View top cat:** `PEEK`
- **Serve top-priority cat:** `SERVE`
- **Change mode**
  - `MODE ADOPTION`
  - `MODE TRIAGE`

- **Set/reset featured breed**
    - `FEATURED <breed> <alpha>`
    - `FEATURED NONE 1.0`
- **Print top k (non-destructively):** `PRINT <k>`
- **Quit:** `QUIT`

Required Data Structures and Function Prototypes

Do not rename, remove, or alter any required fields of the provided structures. You may add additional fields or helper structs only if they don't break these definitions or the specified behaviors. The provided function prototypes are also mandatory. You may write additional helper functions, but do not alter or overload these required prototypes.

```
/* ---------- Global Modes ---------- */
typedef enum {
    MODE_ADOPTION = 0,  /* Max-heap by adoption_key (higher is better)   */
    MODE_TRIAGE   = 1   /* Min-heap by triage_key   (lower is more urgent)*/
} Mode;
/* ---------- Core Cat Record ---------- */
typedef struct Cat {
    char *name;          /* dynamically allocated, unique string (<= 25 chars +
'\0') */
    char *breed;         /* dynamically allocated string */
    int age;             /* years */
    int friendliness;    /* 0..100 */
    int health;          /* 0..100      (higher means healthier) */
    int arrival_id;      /* strictly increasing when ADDed */
    int quarantine;      /* 0 or 1 (1 => cannot be adopted; S1) */
    double key;          /* cached priority value for the **active** mode */
} Cat;
/* ---------- Array-Based Binary Heap of Cat* ---------- */
typedef struct {
    Cat **arr;           /* array of Cat* implementing the heap */
    int size;            /* current number of elements */
    int capacity;        /* allocated capacity */
    Mode mode;           /* controls comparator semantics */
} CatHeap;
/* ---------- Global Shelter State ---------- */
typedef struct {
    Mode mode;           /* MODE_ADOPTION or MODE_TRIAGE */
    char *featured_breed;/* NULL => none; else dynamically allocated breed string
*/
    double alpha;        /* multiplier for featured breed (>= 1.0; default 1.0)*/
    int next_arrival_id; /* increments on each ADD */
    CatHeap heap;        /* priority structure */
} Shelter;
/* Returns heap index of cat with given name, or -1 if not found */
int find_cat_index(CatHeap *heap, char *name);
/* Returns the current adoption-mode key for cat c using S->featured_breed and S-
>alpha. */
double compute_adoption_key(Cat *c, Shelter *S);
/* Returns the current triage-mode key for cat c. */
double compute_triage_key(Cat *c);
/* Recomputes all keys for the active mode and rebuilds heap in O(n) using bottom-
up heapify. */
void recompute_all_keys_and_build(Shelter *S);

/* ========== Command Handlers (I/O-Free Logic) ========== */
/* Allocates a new Cat, initializes fields, computes key for active mode,
   ensures no duplicate name exists (linear scan), and inserts into the heap. */
```

```
void cmd_add (Shelter *S, char *name, char *breed, int age, int friendl, int
health);
/* Locate the cat by name using a linear scan of the heap array.
   If found, update the requested field.
   For AGE/FRIEND/HEALTH: recompute key for active mode and restore heap order.
   For QUARANTINE: update the flag only (numeric key unchanged), then restore heap
order. */
void cmd_update(Shelter *S, char *name, char *field, int new_value);
/* Locate the cat by name using a linear scan of the heap array.
   Remove it from the heap, restore heap order, and free the Cat. */
void cmd_remove(Shelter *S, char *name);
/* Prints the current top cat for the active mode (does not modify heap). */
void cmd_peek (Shelter *S);
/* Serves the highest-priority cat based on the active mode.
   ADOPTION: adopt highest-priority non-quarantined cat.
   TRIAGE: treat most urgent cat. */
void cmd_serve(Shelter *S);
/* Sets S->mode, sets heap mode, then recompute_all_keys_and_build(S). */
void cmd_mode  (Shelter *S, char *mode_str);
/* Sets (or clears) featured breed and alpha, then
recompute_all_keys_and_build(S). */
void cmd_featured(Shelter *S, char *breed, double alpha);
/* Non-destructive: print top k according to active mode.
   Recommended: copy heap array into a temp heap and extract k from the copy. */
void cmd_print (Shelter *S, int k);
```

Output Specifications

General rules (apply to every line you print)

- Exact text/case/spacing; ASCII; no extra blank lines.
- Keys and alpha: two decimals (rounded).
- Follow per-command patterns as shown.

1) Mode Changes, print: `Mode set to <MODE>. Rebuilding priorities...`

Only `ADOPTION` and `TRIAGE` are valid modes. For all other entries, print: `Unknown mode <unknown>.`

2) Featured Breed:

A. Set or change to a breed, print: `Featured breed set to <breed> with alpha=<alpha_formatted>. Rebuilding priorities...`

- `<alpha_formatted>`: print with two decimals (e.g., 1.10, 1.25).

B. Clear featured breed, print: `Featured breed cleared. Rebuilding priorities...`

3) Adding Cats:

- On success, print: `Added <name>.`
- If a cat with the same name already exists, print: `Name <name> already exists.`

Note: ADD never prints keys or mode tags.

4) Peek Top Cat:

- If the heap is empty, print: `No cats available.`
- Otherwise, print one line describing the current top **for the active mode**:
  `Top[<MODE>]: [<MODE>] (key=XX.YY, name=<name>, breed=<breed>, age=<age>, friend=<friendliness>, health=<health>)`

Formatting details:

- Exactly one space after the colon :.
- The inner tag [ADOPTION] or [TRIAGE] must match the mode in the brackets after Top[...].
- key is two decimals.
- Field order and commas exactly as shown.

- Quarantine does not change PEEK behavior; quarantined cats may appear here.

5) Serve Cat (SERVE):

If the heap is empty, print: `No cats available.`

If current mode is ADOPTION:

- If all cats are quarantined, print: `No adoptable cats available.`
- Otherwise, the highest-priority non-quarantined cat is adopted, print one line:
  `Serve now: <name> (key=XX.YY, name=<name>, breed=<breed>, age=<age>, friend=<friendliness>, health=<health>)`

If current mode is TRIAGE:

- The most urgent cat is treated, print:
  `Serve now: <name> (key=XX.YY, name=<name>, breed=<breed>, age=<age>, friend=<friendliness>, health=<health>)`

Note:

- Only one cat is served per SERVE command.
- Quarantine blocks adoption but does not affect TRIAGE.

6) Update Cat:

- If <name> doesn't exist, print: `Cat <name> not found.`
- For FIELD in {AGE, FRIEND, HEALTH}:
  Update the field, recompute key according to active mode, adjust heap position, then print:
  `Updated <name>: <FIELD>=<value>. Priority adjusted.`
  <FIELD> must match exactly one of AGE, FRIEND, HEALTH.
- For `FIELD == QUARANTINE`:
  Accept 0 or 1; do not change the numeric key; adjust position (safe no-op) and print:
  `Updated <name>: QUARANTINE=<0_or_1>.`
- For any other field, print:
  `Unknown field <FIELD>.`
- Breed is **immutable** after `ADD`; `UPDATE` does not support changing breeds.

7) Remove Cat:

- If <name> doesn't exist, print: `Cat <name> not found.`
- On success, print: `Removed <name>.`

8) Print Top-k (Non-Destructive):

- If heap is empty, print: `No cats available.`
- Otherwise, print up to k lines (or fewer if fewer than k cats exist). Each line shows the rank from 1:

  `[<rank>] <name> (key=XX.YY, <MODE>)`

  where `<MODE>` represents the active mode.

Formatting details:

- Square brackets around the 1-based rank.
- One space after the closing bracket.
- Exactly one space after the comma before the mode label.
- Do not include breed/age/friend/health here, only the name, key, and mode tag.
- **Note:** The heap must not change as a result of PRINT.

9) Quit: Print nothing for QUIT. Program terminates after processing QUIT or after reading all q commands.

10) Formatting & Rounding Rules

- Priority keys (key=XX.YY) and Alpha in Featured line: Always two decimals, rounded using standard rounding (e.g., 1.234 → 1.23, 1.235 → 1.24). No thousand separators.
- Integers (age, friend, health, counts, ranks, total): Print as plain base-10 integers; no leading zeros.

- Exactly one space after colons : and commas , where shown. Do not add trailing spaces at the end of lines.

Runtime and Memory Requirements

Runtime:

- Insert: O(log n)
- Update by name: O(n)
- Remove by name: O(n)
- Serve: O(log n)
- Mode / Featured change: O(n)
- PRINT k: O(k log n) using a temporary heap
- n ≤ approximately 5,000 cats

Memory:

- All strings dynamically allocated
- Free all dynamically allocated memory (cats, strings, heap, featured breed)
- No memory leaks permitted

Sample I/O

Sample Input	Sample Output
13 MODE ADOPTION ADD Coco Ragdoll 4 88 90 ADD Hana DSH 3 92 85 UPDATE Coco QUARANTINE 1 UPDATE Hana QUARANTINE 1 SERVE UPDATE Hana QUARANTINE 0 SERVE MODE TRIAGE ADD Milo Siamese 15 40 60 SERVE PRINT 2 QUIT	Mode set to ADOPTION. Rebuilding priorities... Added Coco. Added Hana. Updated Coco: QUARANTINE=1. Updated Hana: QUARANTINE=1. No adoptable cats available. Updated Hana: QUARANTINE=0. Serve now: Hana (key=238.60, name=Hana, breed=DSH, age=3, friend=92, health=85) Mode set to TRIAGE. Rebuilding priorities... Added Milo. Serve now: Coco (key=15.60, name=Coco, breed=Ragdoll, age=4, friend=88, health=90) [1] Milo (key=81.00, TRIAGE)

Implementation Hint:

- You are NOT required to use any tree or map structure.
- To find a cat by name, simply scan the heap array.
- Focus on correctly restoring the heap after updates or removals.

Implementation Requirements:

- Your program must be written in C and must compile on Eustis.
- Make sure to submit well-structured and well-commented code to avoid penalties.
- No global variables except those explicitly required.
- You may not use C++ features, external libraries beyond standard C libraries such as qsort, priority_queue, map, etc., system() calls, or any OS-specific calls. Such approach will result in a -100 penalty.
- You may not use multiple .c or .h files, preprocessor tricks that alter required structures, or hard-coded output for any scenario.
- All input must be read using scanf. All output must be printed using printf exactly as specified in the "Output Specifications" section. No file I/O, including: fopen(), fprintf(), fscanf(), fgets(), etc. No additional or missing newlines.

- All cat names and breed strings must be read into a temporary static buffer first, then dynamically allocated with the exact amount of memory needed, and finally copied using strcpy.
- All memory must be dynamically allocated using malloc, calloc, or realloc.
- You must not: Use global static arrays, Use fixed-length arrays for cat storage, Leak memory in any scenario.
- At the end of execution (before quitting), you must free: Every remaining cat, All name and breed strings, everything that had memory allocated. When removing a cat, you must free: cat->name, cat->breed, and the Cat* object itself.

Deliverables:

- Please submit the source file, `cat_heap.c`, via Webcourses.